

1. Pelajari artikel pada link: (<https://www.analyticsvidhya.com/blog/2022/07/understanding-transaction-management-in-sql/>)

Berdasarkan pemahaman anda, jawablah pertanyaan berikut:

- a) Bagaimana penerapan scheduling dapat berdampak pada terjaganya konsistensi data?

Answer:

Scheduling menyatukan beberapa transaksi dalam 1 batch dan mengeksekusinya dalam urutan yang spesifik. Scheduling menjaga konsistensi data dengan tidak melakukan lebih dari 1 transaksi bekerja secara bersamaan.

- b) Berikan contoh transaksi yang membutuhkan penerapan Serial Scheduling!

Answer:

Transaksi T1:

Baca(x)

$x = x + 5$

Tulis(x)

Transaksi T2:

Baca(x)

$x = x * 2$

Tulis(x)

- c) Berdasarkan contoh yang diberikan pada poin B, berikan analisa dampak yang disebabkan jika transaksi tersebut tidak diatur menggunakan Serial Scheduling.

Answer:

Jika tidak menggunakan serial scheduling maka x yang seharusnya ditambah 5 terlebih dahulu baru di kali dengan 2, bisa terjadiny sebaliknya atau hanya ditambah saja atau di kali saja.

2. Jelaskan fungsi **Cursor** pada SQL?

Answer:

Cursor adalah sebuah tempat kerja sementara yang terjadi saat melakukan SQL statement. Cursor ini berisi tentang statement yang digunakan dan data row yang diakses oleh statement tersebut. Data row yang diakses akan dimanipulasi sesuai SQL statement. Intinya, cursor digunakan untuk mengakses baris data dan memprosesnya sesuai query yang di masukan.

3. Berikan contoh pada kasus seperti apa **Cursor** perlu diterapkan!

Answer:

Misal kita memiliki table "Produk" dan ingin memprint nama produk dan harga pada setiap produk.

4. Tuliskan contoh query deklarasi **Cursor**, dan cara mengakses data dari **Cursor** tersebut.

Answer:

```
OPEN cursor_product; # Open Cursor
```

```
FETCH NEXT FROM cursor_product INTO product_name, list_price; # Go to the 1st row
```

```
WHILE @@FETCH_STATUS = 0; # While row is not full
```

```
BEGIN # Begin while process
```

```
PRINT product_name + ' ' + CAST(list_price AS VARCHAR); # Print
```

```
FETCH NEXT FROM cursor_product INTO product_name, list_price; # Go to next row
```

END; # End while proses

CLOSE cursor_product; # Close Cursor

DEALLOCATE cursor_product; # Deallocate cursor_product from memory

5. Suatu aplikasi kasir menyimpan data pelanggan, produk serta transaksi produk yang dibeli oleh pelanggan. Contoh data yang disimpan adalah sebagai berikut,

Tabel Customers

<u>CustomerID</u>	FirstName	LastName	Email
1	John	Doe	john.doe@gmail.com
2	Jane	Smith	jane.smith@yahoo.com
3	Bob	Johnson	bob.johnson@yahoo.com
4	Alic	Williams	alice.w@outlook.com

Tabel Products

<u>ProductID</u>	ProductName	Price	Quantity
201	Laptop	1200.00	10
202	Smartphone	500.00	25
203	Headphones	80.00	50
204	Printer	300.00	15
205	Tablet	350.00	20

Tabel Orders

<u>OrderID</u>	<u>CustomerID</u>	<u>ProductID</u>	OrderDate	Quantity	TotalAmount
1	1	201	2023-05-10	2	2400.00
2	2	202	2023-05-10	1	500.00
3	3	203	2023-05-10	5	400.00
4	1	204	2023-05-10	1	300.00
5	2	205	2023-05-11	3	1050.00
6	3	201	2023-05-11	2	2400.00
7	1	203	2023-05-11	1	80.00
8	2	204	2023-05-12	2	600.00

Berdasarkan contoh table dan data yang diberikan, lakukanlah hal berikut:

- a. Tuliskan query untuk menampilkan data FirstName, LastName, dan Email yang memiliki alamat email dengan domain @yahoo.com.

Answer:

```
SELECT FirstName, LastName, Email FROM Customers WHERE Email LIKE “%@yahoo.com”;
```

- b. Tuliskan query untuk menampilkan data seperti berikut,

FirstName
John
Jane
Alic

Answer:

```
SELECT FirstName FROM Customers;
```

- c. Tuliskan query untuk menghitung Total Jumlah Produk yang Dibeli oleh Setiap Pelanggan.

Answer:

```
SELECT CustomerID, COUNT(Quantity) FROM Orders GROUP BY CustomerID;
```

- d. Tuliskan query untuk menampilkan data seperti berikut,

OrderDate	Sales	Average_Sales
2023-05-10	3600.00	900.000000
2023-05-11	3530.00	1176.666667
2023-05-12	600.00	600.000000

```
SELECT OrderDate, SUM(TotalAmount) AS "Sales", AVERAGE(Average_Sales) AS  
"Average_Sales" FROM Orders GROUP BY OrderDate;
```

- e. Tuliskan query untuk menampilkan nama produk dan jumlah produk yang sudah terjual, dengan kriteria tambahan hanya data dengan produk terjual lebih dari 3 yang akan ditampilkan.

ProductName	NUMBER_OF_SOLD
Headphones	6
Laptop	4

```
SELECT p.ProductName, COUNT(o.Quantity) AS 'NUMBER_OF_SOLD' FROM Orders o INNER  
JOIN Products p ON o.ProductID = p.ProductID GROUP BY ProductID;
```

- f. Tampilkan data order yang memiliki jumlah produk (Quantity) lebih besar dari nilai rata-rata jumlah produk yang telah diorder.

```
SELECT * FROM Orders WHERE Quantity > AVERAGE(Quantity);
```

- g. Tampilkan data produk yang telah terjual dan harganya di atas nilai rata-rata harga produk.

ProductName	Price
Laptop	1200.00
Smartphone	500.00

```
SELECT ProductName, Price FROM Products WHERE Price > AVERAGE(Price)
```

- h. Tuliskan query pembuatan trigger untuk mengurangi stock secara otomatis ketika terjadi order oleh customer

```
CREATE TRIGGER reduc_stock_after_order AFTER INSERT ON Orders FOR EACH ROW  
BEGIN
```

```
UPDATE Products
SET stock = stock – NEW.Quantity
WHERE ProductID= NEW.ProductID
END;
```